

# 교수님 캘리브레이션 피드백 완전 해설

녹음 전사본에서 나온 6개 지적을 “무엇을 걱정하신 것인가 → 코드가 실제로 무엇을 하는  
가 → 그래서 어떻게 답하면 되는가” 순서로 풀어 쓴 학습용 자료.

모든 수식·코드는 `rb-calibration-marker-experiment` 저장소의 실제 구현에서 그대로 가져왔다.

session02 · canonical run | split seed 20260731 | held-out events [29, 38, 46, 50, 53, 61, 69, 75, 92]

Cube-in-Hand Calibration · 고정카메라 3대 + 그리퍼카메라 1대

# 이 자료를 읽는 방법

6개 쟁점 · 47장 · 각 장은 “걱정 → 수식 → 코드 → 답변”으로 읽는다

## 슬라이드마다 같은 4단 구조

|      |                                            |
|------|--------------------------------------------|
| ① 걱정 | 교수님이 실제로 의심한 지점. 대개 “네 방법이 이기도록 설계된 것 아니냐” |
| ② 수식 | 그 의심을 수학적으로 정확히 쓰면 무엇인가                    |
| ③ 코드 | 저장소의 실제 구현 (노란 강조줄이 핵심)                    |
| ④ 답변 | 면담에서 그대로 말할 수 있는 문장                        |

## 가장 중요한 한 문장

6개 지적은 따로 노는 게 아니라 하나의 질문이다 — “평가가 네 방법에 유리하게 설계돼 있지 않은가?”  
 대응 원칙도 하나다: 평가 경로에서 그 방법이 쓴 정보를 빼고, 모든 방법에 같은 데이터·같은 mask를 강제하고, 아직 못한 건 코드에 ‘못했다’고 박아둔다.

## 목차

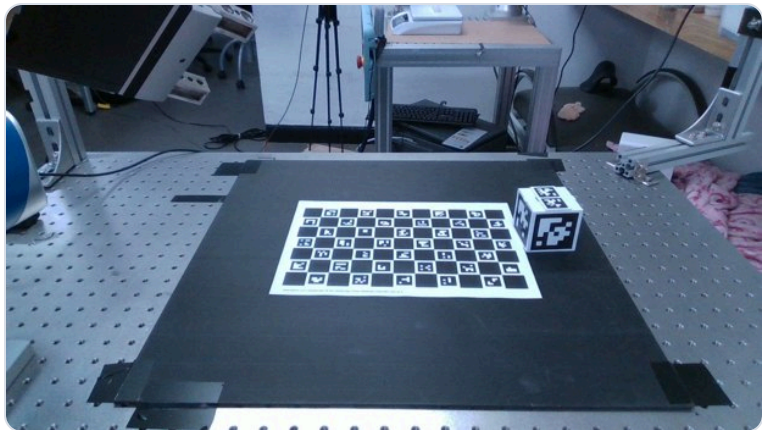
- **Part 1** · 독립 외부 GT — 추후
- **Part 2** · FK 순환 논증 — 수정 완료 ← 승부처
- **Part 3** · 목적함수 · 단위 — 수정 완료
- **Part 4** · 동일 데이터 · 동일 기준 — 수정 완료
- **Part 5** · 외부 baseline — 추후
- **Part 6** · 아웃라이어 — 수정 완료
- **Part 7** · pose 좌표계 혼용 — 추가 발견
- **Part 8** · 예상 재질문 대비 · 면담 순서

## 읽을 때 유의

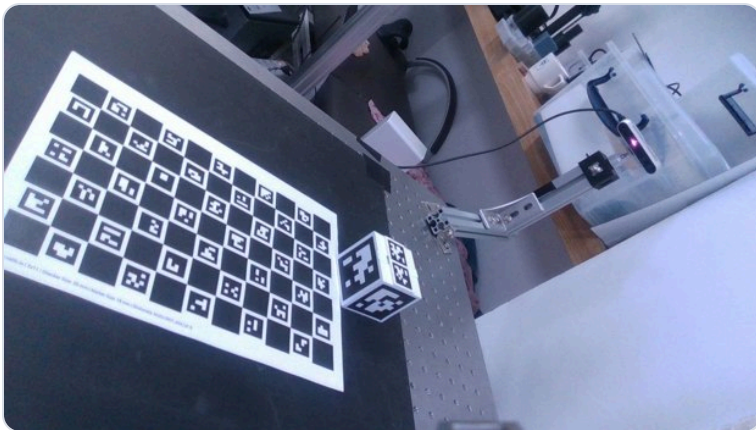
Part 2와 Part 8의 **Q1(11.5 mm 문제)**만 확실히 준비하면 나머지는 문서 내용으로 커버된다.

# 실험 셋업 — 무엇을 찍고 있는가

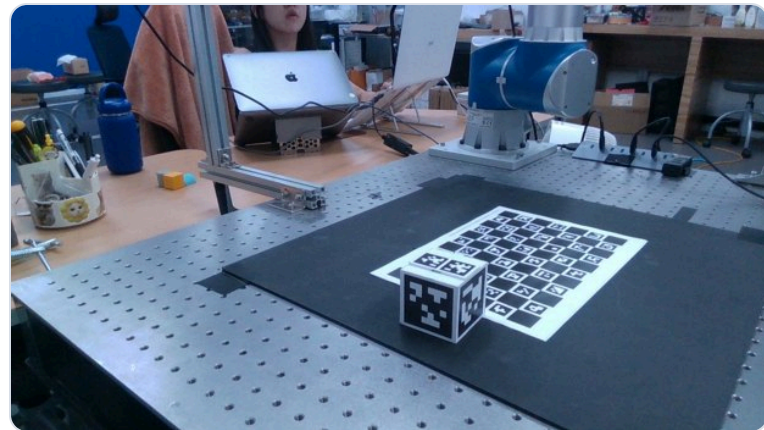
session02 · event 29 (held-out) 의 동일 순간을 세 카메라가 본 모습



cam0 · 고정카메라 (eye-to-hand)



cam2 · 고정카메라 (eye-to-hand)



cam3 · 그리퍼카메라 (eye-in-hand)

## 두 개의 표적

**ArUco 큐브** (DICT\_4X4\_50, 큐브 30 mm / 마커 22 mm) — 로봇이 잡아서 옮기는 3D 표적. FK가 붙는 곳.

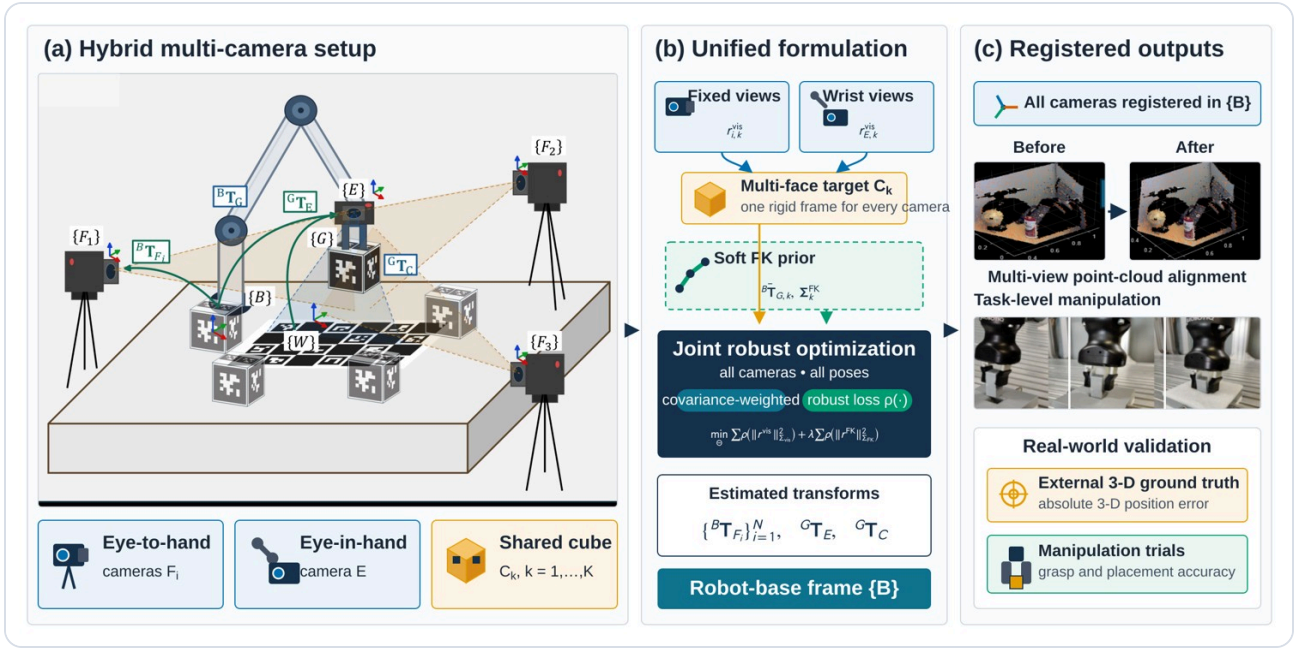
**ChArUco 보드** (DICT\_4X4\_250, 7×11, 체커 22 mm / 마커 16 mm) — 평면 표적.

## 왜 큐브를 “잡는가”가 핵심인가

로봇이 큐브를 잡으면 FK가 큐브 pose를 알려준다. 즉 원래는 미지수였던  ${}^B T_O$  를 기지(既知) 구속으로 바꿀 수 있다 — 이게 논문의 novelty이자, 교수님이 “그럼 순환 아니냐”고 의심하는 바로 그 지점이다.

# 기호 정리 — 이 자료에서 쓰는 좌표계와 변수

이후 모든 수식은 이 기호만 사용한다



논문 main figure — (a) 셋업 (b) 통합 정식화 (c) 등록 결과

| 기호                   | 의미                                                                |
|----------------------|-------------------------------------------------------------------|
| $B$                  | robot base frame — 모든 것이 최종적으로 등록되는 기준                            |
| $G$                  | gripper (flange) frame                                            |
| $C_i$                | 고정카메라 $i$ ( $i = 0, 1, 2$ )                                       |
| $C_g$                | 그리퍼(손목) 카메라                                                       |
| $O$                  | target — 큐브 또는 보드                                                 |
| ${}^B T_G(e)$        | <b>robot FK — 이벤트 <math>e</math>에서 측정되는 상수 (모든 조건에서 backbone)</b> |
| ${}^B T_{C_i}$       | 고정카메라 외부파라미터 ( <b>미지</b> )                                        |
| ${}^G T_{C_g}$       | hand-eye 변환 ( <b>미지</b> )                                         |
| ${}^B T_O$           | target pose ( <b>미지 또는 FK로 고정</b> ) ← 쟁점의 핵심                      |
| $K_i, D_i$           | 내부 파라미터 · 왜곡 ( <b>상수, 최적화 변수 아님</b> )                             |
| $X_n^O$              | target 좌표계에서의 $n$ 번째 코너 3D 위치                                     |
| $\mathbf{u}_{i,e,n}$ | 실제로 검출된 2D 픽셀 코너 ( <b>관측값</b> )                                   |

# 교수님 피드백 6개의 공통 뿌리

이 한 장이 면담 전체의 프레임이다

여섯 개는 결국 하나의 질문이다

“네 방법이 이기도록 평가가 설계되어 있는 것 아니냐?”

## 갈래 A — 순환 논증

쟁점 1, 2

정답(GT)이 네 방법의 입력과 같은 출처면 그건 검증이 아니라 자기확인이다.

가장 위험

## 갈래 B — 비교 불공정

쟁점 3, 4, 5

방법마다 데이터·평가기준·단위가 다르면 표의 순위 자체가 무의미하다.

방어 튼튼

## 갈래 C — 사후 선별

쟁점 6

결과를 보고 나쁜 데이터를 빼면 무엇이든 좋아진다.

방어 가장 깔끔

## 면담 오프닝으로 쓸 문장

“지적해 주신 여섯 가지를 하나의 원칙으로 정리했습니다 — 평가 경로에서 그 방법이 사용한 정보를 제거하고, 모든 방법에 동일한 데이터와 동일한 사전 mask를 강제하고, 아직 수행하지 못한 검증은 코드가 ‘미수행’으로 표시하도록 만들었습니다.”

PART 1 · 쟁점 1

# 독립된 외부 GT가 필요한가?

전사 위치 07:39-09:31, 54:13-56:02

결론: 반박하지 않는다. 인정하고 넘어간다. 대신 코드가 절대정확도라는 표현 자체를 못 쓰게 막았다.

# 순환 검증이란 정확히 무엇인가

“FK나 영상으로 되만든 값을 정답으로 두지 말라”는 지적의 정확한 의미

## 지금 구조 — 정답이 내부에서 나온다



FK로 만든 값도, 영상으로 되만든 값도 전부 내 시스템의 산출물이다. 이 안에서 아무리 잘 맞아도 “물리적으로 1 mm 정확하다”는 주장은 성립하지 않는다.

## 있어야 하는 구조 — 독립 측정계



calibration\_pipeline/table1.py · L1440–1445

```

"e_task_pose": {
  "available_external_ground_truth": False,
  "allowed_current_label": TASK_POSE_PROXY_LABEL,
  "may_be_described_as_absolute_accuracy": False,
  "protocol": "separate_position_holdout_not_event_holdout",
},
  
```

## 코드가 하는 일

결과 protocol에 “외부 GT 없음 / 절대정확도로 기술 금지”를 강제로 기록한다. 즉 나중에 누가 이 결과를 절대정확도처럼 쓰려 해도 파일 자체가 반박한다.

## 덤으로 알아둘 것

$e_{\text{cross}}$ ,  $e_{\text{e2e}}$ , 공통 target 재투영은 전부 **internal consistency / internal transfer** 지표다. “일관성이 좋다”와 “정확하다”는 다른 말.

# 답변 카드 — 쟁점 1

## 이대로 말하면 되는 문장

“현재  $e_{\text{cross}}$ ,  $e_{\text{e2e}}$ , 공통 target 재투영은 모두 내부 일관성 또는 내부 전이 성능이지 독립적인 절대 3D 정확도가 아닙니다. 그래서 결과 protocol에 절대정확도로 기술 금지 플래그를 넣어 두었고, 독립 지그 기반 blind position holdout은 별도 항목으로 남겨 두었습니다.”

## 전략적 의미 — 왜 이걸 제일 먼저 꺼내야 하나

먼저 인정하면 나머지 5개 답변의 신뢰도가 올라간다. 방어부터 시작하면 전체가 변명으로 들리고, 인정부터 시작하면 나머지가 근거로 들린다.

## 현재 상태

코드

플래그로 차단 완료

실험

추후 진행

필요한 것

눈금 큐브 지그 / 독립 측정 기준  
+ blind position holdout 취득

## 말하면 안 되는 것

- “재투영 오차가 2 px니까 정확합니다”
- “ $e_{\text{cross}}$ 가 SOTA보다 좋습니다”
- 내부 지표를 mm로 환산해 SOTA와 나란히 놓기



PART 2 · 쟁점 2

# FK를 정답처럼 쓰면 FK 계열이 유리한 것 아닌가?

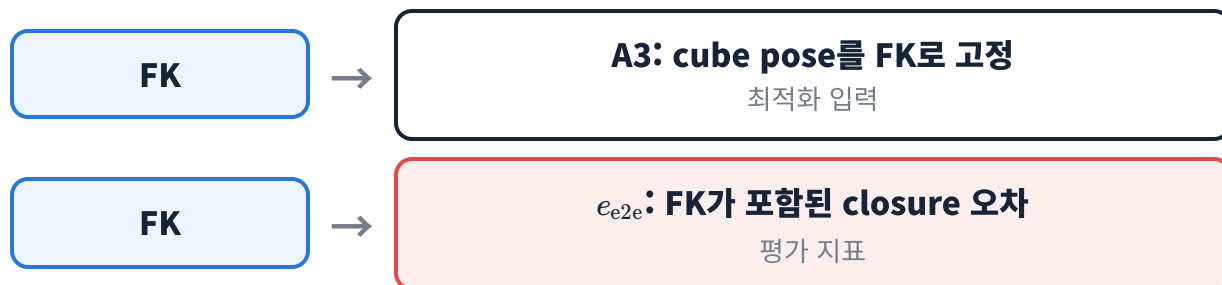
전사 위치 11:46-12:26, 34:18-35:00, 54:13-56:02

이 미팅의 진짜 승부처. 리뷰어가 100% 짚는 자리이고, 논문 리젝 사유가 될 수 있는 급이다.

# 왜 이것이 가장 아픈 지적인가

네 novelty와 네 평가 지표가 같은 정보를 쓰고 있다

## 동어반복의 구조



FK를 쓴 방법이 FK 기반 오차에서 좋은 건 **당연한 결과**이지 발견이 아니다. novelty가 “FK로 미지수를 제거한다”인데 평가에도 FK가 있으면, 주장과 증거가 같은 것을 가리킨다.

## 따라오는 두 번째 질문

“FK가 부정확한 **저가** 로봇에서도 같은 결론이 나오는가?”

→ A3는 FK 오차를 그대로 bias로 받는다. 이 질문에 대한 정직한 답은 “**아직 모른다**”이다.

## 방어는 두 겹으로 짜여 있다

### ① 조건 분리

FK 사용 강도를 **실험** 축으로 만든다 — A2(안 씀) / A3(하드 고정) / A4(soft prior)



### ② FK 없는 평가 지표 $e_{cross}$

A3에게 **아무 이점이 없는** 링에서 A3를 다시 세운다

## 핵심 감각

“FK를 안 썼다”고 우기는 게 아니다. **FK를 쓴 정도를** 축으로 만들고, FK가 없는 자를 하나 더 들고 온다.

# ① 조건 분리 — FK 사용 강도를 축으로

A2 / A3 / A4는 “다른 알고리즘”이 아니라 FK 사용 강도만 바꾼 같은 파이프라인이다

| 조건            | cube pose 취급            | FK 의존 |
|---------------|-------------------------|-------|
| A2            | 자유변수로 추정<br>vision only | 없음    |
| A3            | FK로 하드 고정<br>변수에서 제거    | 최대    |
| A4<br>B1 · B2 | 자유변수 + FK soft prior    | 중간    |

## 왜 A3가 held-out에서 유리할 수 있나

A3는 자유도가 가장 적다. 관측 대비 파라미터가 줄면 과적합이 줄어 일반화가 좋아진다.

단, FK가 틀리면 그 오차가 그대로 bias로 박힌다 — 이게 저가 로봇 질문의 정확한 의미.

calibration\_pipeline/schema.py · L52–67 (실제 조건 정의)

```

MAIN_ABLATION_CONDITIONS = (
    AblationCondition("A0", "board", "seq", "-", "estimated", "baseline"),
    AblationCondition("A1", "cube+board", "seq", "estimated", "estimated", "+cube"),
    AblationCondition("A2", "cube+board", "U", "estimated", "estimated", "+unified"),
    AblationCondition("A3", "cube+board", "U", "FK-fixed", "estimated", "Ours (full)"),
    AblationCondition(
        "A4", "cube+board", "U", "corrected-FK-factor", "estimated",
        "Ours (corrected-FK factor)",
    ),
    AblationCondition(
        "B1", "cube+board", "seq", "corrected-FK-factor", "estimated",
        "-Unified",
    ),
    AblationCondition("B2", "cube", "U", "corrected-FK-factor", "-", "-board"),
    AblationCondition("B3", "board", "U", "-", "estimated", "-cube"),
)

```

## 표를 읽는 법

target\_set = 어떤 마커를 쓰는가 · unified = 순차(seq)/통합(U) · fk\_to\_cube = cube pose를 무엇으로 두는가 ← 이 열이 쟁점 2의 축이다.

# A2와 A3의 차이는 코드 한 줄이다

“A3만 다른 경로로 푼 것 아니냐”에 대한 코드 수준의 답

calibration\_pipeline/schema.py · L169–175 (조건별 자유변수)

```
UNIFIED_FREE_VARIABLES: Mapping[str, tuple[str, ...]] = {
    "A2": ("T_base_Ci", "T_gripper_cam", "T_base_board", "T_base_cube_by_set"),
    "A3": ("T_base_Ci", "T_gripper_cam", "T_base_board"),
    "A4": ("T_base_Ci", "T_gripper_cam", "T_base_board", "T_base_cube_by_set"),
    "B2": ("T_base_Ci", "T_gripper_cam", "T_base_cube_by_set"),
    "B3": ("T_base_Ci", "T_gripper_cam", "T_base_board"),
}
```

## 여기가 “hard-FK”의 전부다

A2에는 있고 A3에는 없는 항목: "T\_base\_cube\_by\_set"

즉 A3는 별도의 알고리즘이 아니라, 같은 solver를 자유변수 하나 빼고 돌린 것이다. 이 사실이 “A3만 특별대우 받은 것 아니냐”는 의심을 정면으로 반박한다.

calibration\_pipeline/table1.py · L540–549

```
elif condition.row == "A3":
    # Hard-FK is represented solely by the freeze mask: cube transforms
    # are present in the state but absent from A3's declared free
    # variables.
    final_state, diag = solve_stage(
        relevant_train, UNIFIED_FREE_VARIABLES["A3"], initial_state,
        robot_T, K_map, D_map, gripper, seed, args)
    diag["entry_point"] =
    "calibration_pipeline.reprojection.solve_corner_reprojection"
    diag["fk_mode"] = "fixed"
    diag["shared_reference_state_sha256"] = state_sha256(initial_state)
    stages = {"joint_eih_e2h": diag}
```

## 주석이 말하는 것

*“Hard-FK is represented solely by the freeze mask: cube transforms are present in the state but absent from A3’s declared free variables.”*

cube pose는 state 안에 존재하지만 자유변수 목록에 없다. A3도 다른 통합 조건과 완전히 같은 진입점 ( solve\_corner\_reprojection )을 호출한다.

## ② soft-FK factor — 수식 (1) 보정된 FK

A4 / B1 / B2가 쓰는 “corrected FK”가 무엇인지부터

raw FK에 train에서만 추정된 보정을 곱한다

$$\tilde{T}_{O,s}^B = T_{O,s,FK\text{ raw}}^B \Delta_{\text{train}}$$

### 세 가지만 이해하면 된다

- $s = \text{set}$  (큐브를 한 번 놓아둔 구간). 그 구간 동안 큐브는 정지 상태.
- $\Delta_{\text{train}}$  = FK가 체계적으로 어긋난 양을 **train 데이터만으로** 추정한 보정. → held-out을 보지 않으므로 **test 누수 없음**.
- $\tilde{T}$  = “보정된 FK(corrected FK)”. 이게 soft factor의 **중심**이 된다.

### 누수 검사도 코드에 있다

이 artifact가 held-out event를 하나라도 썼으면 실행이 중단된다

calibration\_pipeline/table1.py · L1057–1062 (누수 차단)

```
{
    leaked_test_events = (
        set(artifact["source_observation_ids"]) &
        set(split["test_events"]))
    if leaked_test_events:
        raise RuntimeError(
            f"board-free FK artifact used held-out events
{sorted(leaked_test_events)}")
    leaked_raw_events = {
```

calibration\_pipeline/schema.py · L27–39 (계약 선언)

```
"aligned_pose": "T_fk_aligned_set = T_fk_raw_set @ Delta_train",
"delta_estimation": (
    "board_free_training_eye_in_hand_cube_corner_reprojection"),
"cube_pose_is_optimization_variable": True,
"factor_residual": (
    "r_set=[Log_S03(R_cube^T R_fk_aligned), "
    "translation(inv(T_cube)@T_fk_aligned)]"),
"whitening": "w_set=chol(Sigma_set)^(-1)@r_set",
"objective":
"robust_visual_corner_residuals_plus_robust_whitened_FK_factors",
"hard_gate_or_pose_replacement": False,
"external_ground_truth_used": False,
}
```

## ② soft-FK factor — 수식 (2) 잔차와 whitening

회전 rad과 병진 m을 어떻게 한 목적함수에 넣는가

① 추정 cube pose와 보정 FK pose의 SE(3) 차이

$$\xi_s = \begin{bmatrix} \text{Log}\left(R_{O,s}^\top \tilde{R}_{O,s}\right) \\ t\left((T_{O,s}^B)^{-1} \tilde{T}_{O,s}^B\right) \end{bmatrix} \in \mathbb{R}^6$$

② 불확실성으로 정규화(whitening) —  $\Sigma_s = L_s L_s^\top$  (Cholesky)

$$w_s = L_s^{-1} \xi_s \quad \implies \quad \|w_s\|^2 = \xi_s^\top \Sigma_s^{-1} \xi_s$$

### ★ 이 슬라이드에서 챙길 단 하나

고전적 질문 “회전(rad)과 병진(m)을 무슨 근거로 더하느냐”의 답이 바로  $\Sigma_s$ 다. 임의로 정한 가중치가 아니라 FK 불확실성의 실측 공분산이 단위를 맞춰준다. → 이걸 네가 먼저 꺼내면 점수를 따는 부분.

### Mahalanobis whitening을 그림으로



FK가 잘 아는 방향의 어긋남은 크게 벌하고, 원래 부정확한 방향의 어긋남은 관대하게 넘긴다.

### A3 vs A4를 한 줄로

A3 =  $\Sigma \rightarrow 0$  (FK를 절대 신뢰) · A4 =  $\Sigma$ 가 유한 (FK를 증거로만 신뢰)

## ② soft-FK factor — 코드 원본

calibration\_pipeline/fk\_factor.py · L92–100 (SE(3) 잔차)

```
estimated = np.asarray(estimated, dtype=np.float64)
target = np.asarray(target, dtype=np.float64)
if (estimated.shape != (4, 4) or target.shape != (4, 4)
    or not np.all(np.isfinite(estimated))
    or not np.all(np.isfinite(target))):
    raise ValueError("FK poses must be finite 4x4 matrices")
error = inv_T(estimated) @ target
rotation = Rotation.from_matrix(error[:3, :3]).as_rotvec()
return np.concatenate([rotation, error[:3, 3]]).astype(np.float64)
```

calibration\_pipeline/fk\_factor.py · L236–243 (whitening 적용)

```
estimated = state.cubes[int(set_index)]
target = self.fk_targets[int(set_index)]
if self.fk_spec.mode != FK_MODE_FACTOR:
    raise RuntimeError("factor block requested without mode=factor")
residual = fk_pose_residual(estimated, target)
block = self.whiteners[int(set_index)] @ residual
blocks.append((int(set_index), np.asarray(block, dtype=np.float64)))
return blocks
```

calibration\_pipeline/fk\_factor.py · L245–255 (최종 residual 결합)

```
def factor_residual(self, x: np.ndarray) -> np.ndarray:
    chunks = [
        robustify_elementwise(block, self.fk_spec.loss,
self.fk_spec.robust_scale)
        for _, block in self.factor_raw_blocks(x)
    ]
    return np.concatenate(chunks) if chunks else np.zeros(0, dtype=np.float64)

def residual(self, x: np.ndarray) -> np.ndarray:
    factor = self.factor_residual(x)
    visual = self.visual_residual(x)
    return visual if not len(factor) else np.concatenate([visual, factor])
```

### 읽는 순서

- $\text{fk\_pose\_residual} \rightarrow \xi_s \cdot \text{whiteners}[s] @ \text{residual} \rightarrow w_s = L_s^{-1} \xi_s$
- $\text{robustify\_elementwise}(\cdot, "huber", 3.0) \rightarrow$  FK가 튜 set 하나가 전체를 끌지 못하게
- $\text{concatenate}([visual, factor]) \rightarrow$  시각 잔차와 FK 잔차가 같은 벡터에

주의 — FK factor는 gate도 pose 치환도 아니다. cube pose는 여전히 자유 변수이고, FK는 당기는 힘일 뿐이다.

# 왜 “반복 측정 7회 이상”을 요구하는가

코드가 실측 FK 공분산 파일을 검사하는 방식

calibration\_pipeline/table1.py · L610-620

```
try:
    n_repeats = int(payload.get("n_repeats", 0))
except (TypeError, ValueError):
    n_repeats = 0
# A sample covariance in d=6 dimensions has rank at most N-1. Requiring
# N>=7 is therefore the mathematical minimum for a full-rank 6x6 estimate.
if n_repeats < 7:
    raise ValueError(
        "FK covariance artifact needs at least seven physical repeats "
        "for a potentially full-rank 6D sample covariance")
shared = payload.get("shared_covariance_6x6")
```

## 수학적 이유

$N$ 개 표본으로 만든 표본공분산의 rank는 최대  $N - 1$ 이다.  $\Sigma_s$ 는  $6 \times 6$ 이므로 full rank가 되려면  $N - 1 \geq 6 \Rightarrow N \geq 7$ .

rank가 모자라면  $\Sigma^{-1}$ 이 존재하지 않아 whitening 자체가 정의되지 않는다.

## 지금 상태 — 왜 “preflight”인가

|               |                                          |
|---------------|------------------------------------------|
| 실측 covariance | 없음                                       |
| 현재 사용값        | Simulation prior<br>2.0 mm / 0.30° 등방 대각 |
| 표시            | preflight_simulation_prior               |
| 해당 행          | A4 · B1 · B2                             |

## Q3 대비 (예상 재질문)

“실측을 못 했으면 왜 표에 있냐” → **확증 결과가 아니라 preflight로 라벨링** 돼 있고 **최종 주장에 사용하지 않는다**. 취득 프로토콜(같은 joint command 반복  $N \geq 7$ , 눈금 큐브 지그)까지 말할 수 있으면 훨씬 좋다.



## ② $e_{\text{cross}}$ — FK가 전혀 없는 링

교수님 의심을 무력화하는 핵심 장치

각 고정카메라가 **영상만으로** PnP한 cube pose를 base로 올린다

$${}^B T_O^{(i)} = {}^B T_{C_i} {}^{C_i} T_{O, \text{PnP}}$$

사전 고정된 모든 카메라 쌍에서 서로 얼마나 어긋나는가

$$e_{\text{cross}, t} = \sqrt{\text{mean}_{(i,j)} \left\| t({}^B T_O^{(i)}) - t({}^B T_O^{(j)}) \right\|_2^2}$$

**robot FK**

없음

**gripper camera**

없음

**nominal cube pose**

없음

**외부 GT**

없음

calibration\_pipeline/path\_evaluation.py · L33–43

```
E_CROSS_CONTRACT = {
    "name": "fixed_camera_cube_pose_consistency",
    "translation_metric": "pairwise_cube_center_distance_RMSE_mm",
    "rotation_metric": "pairwise_S03_geodesic_RMSE_deg",
    "per_camera_pose": "T_base_cube_from_camera = T_base_camera @
T_camera_cube_PnP",
    "pair_population": "predeclared_heldout_fixed_camera_pairs",
    {
        "uses_robot_fk": False,
        "uses_gripper_camera": False,
        "uses_nominal_or_ground_truth_cube_pose": False,
        "absolute_accuracy_metric": False,
    }
}
```

### 이 지표의 힘

A3가 유리해질 **어떤 경로도 없다**. 그런데도 A3의  $e_{\text{cross}}$  병진이 가장 작다 (11.5631 mm) — 이게 “FK 덕분에 좋아 보이는 것뿐 아니냐”에 대한 실질적 반증이다.

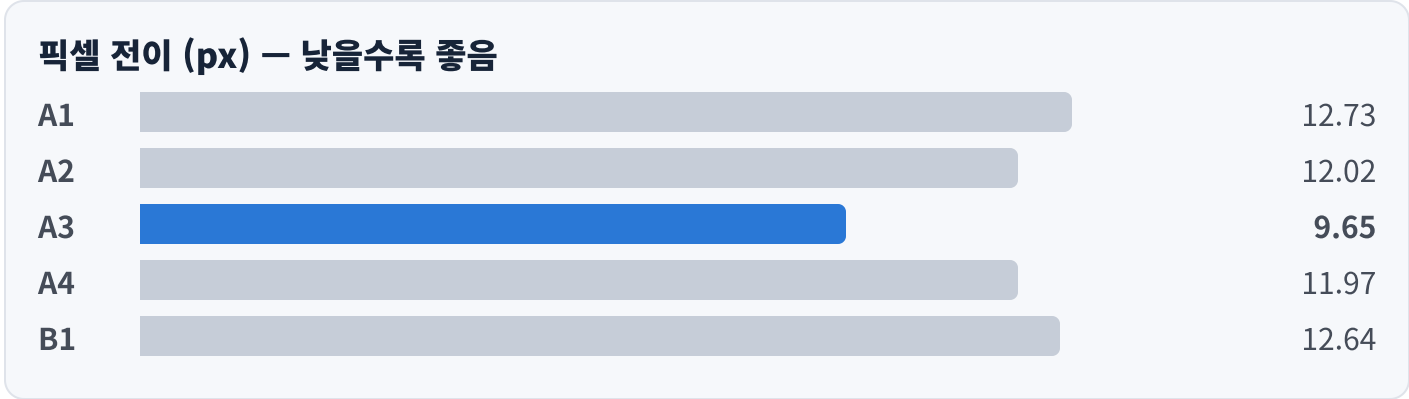
### 한계도 같이 말할 것

$e_{\text{cross}}$  는 모든 카메라가 함께 틀리는 **common-mode** 오차를 검출하지 못한다. 그래서 여전히 절대정확도가 아니다.

# 숫자로 본 답변 — A3는 정말 유리한가

회전 지표는 A3가 지지 않는다 — 이걸 먼저 말해야 신뢰가 생긴다

| 조건                               | 자체 Test px    | 픽셀 전이 px      | $e_{\text{cross}}$ mm | $e_{\text{cross}}$ deg |
|----------------------------------|---------------|---------------|-----------------------|------------------------|
| A1 큐브+보드 · 순차                    | 2.9942        | 12.7282       | 15.8616               | <b>0.8086</b>          |
| A2 큐브+보드 · 통합                    | 2.9715        | 12.0222       | 14.8495               | 0.8437                 |
| <b>A3 통합 · FK-fixed</b>          | <b>2.5315</b> | <b>9.6520</b> | <b>11.5631</b>        | <b>0.8442</b>          |
| A4 통합 · soft FK <b>preflight</b> | 2.9525        | 11.9717       | 14.7545               | 0.8466                 |
| B1 순차 · soft FK <b>preflight</b> | 2.9729        | 12.6368       | 15.7124               | 0.8119                 |



이대로 말하면 되는 문장

“A3를 FK가 들어간  $e_{e2e}$  하나로 평가하지 않습니다. 같은 held-out에서 raw pixel, 카메라 간 양방향 pixel transfer, 그리고 **FK가 전혀 없는**  $e_{\text{cross}}$ 를 함께 봅니다. 현재 A3가 pixel·병진에서 가장 낮지만 **회전 지표는 A1/B1이 더 낮습니다.** 따라서 ‘모든 면에서 우수’가 아니라 ‘**내부 병진·픽셀 기준의 강한 후보**’까지만 주장합니다.”

정직하게 짚어야 할 것

표본이 작다. 공통 관측 **12개 / 229 코너**, 카메라 쌍 6개. 조건 간 0.05 px 차이를 유의하다고 주장하면 안 된다.

PART 3 · 쟁점 3

# “3대를 같이 최적화한다”가 정확히 무슨 뜻인가?

전사 위치 12:47-24:25, 31:40-36:18

교수님이 확인하려던 것: “카메라 3대 PnP 결과를 평균낸 것 아니냐?” (그거면 novelty가 없다)

## 목적함수 — 관측되는 것은 오직 픽셀이다

카메라  $i$ , 이벤트  $e$ , 코너  $n$ 의 예측 픽셀

$$\hat{\mathbf{u}}_{i,e,n} = \pi \left( K_i, D_i, \left( {}^B T_{C_i}(e) \right)^{-1} {}^B T_O X_n^O \right)$$

잔차 — 단위는 픽셀

$$\mathbf{r}_{i,e,n} = \hat{\mathbf{u}}_{i,e,n} - \mathbf{u}_{i,e,n} \quad [\text{px}]$$

두 종류 카메라가 base에 붙는 방식이 다르다

$${}^B T_{C_i}(e) = \begin{cases} {}^B T_{C_i}, & \text{고정카메라 (e2h)} \\ {}^B T_G(e) {}^G T_{C_g}, & \text{손목카메라 (eih)} \end{cases}$$

### “같이 최적화”의 실체 — 연결 고리 3개

① 공유  ${}^B T_O$

서로 다른 카메라가 같은 표적을 본다

②  ${}^G T_{C_g}$

손목카메라는 FK를 타고 base에 붙는다

③  ${}^B T_{C_i}$

각 고정카메라 외부파라미터

### 답변 문장

“PnP 결과를 평균내는 것이 아닙니다. 각 카메라의 원시 2D 코너 잔차가 하나의 최적화 문제에 직접 들어가고, 공유 target/hand-eye 변수를 통해 서로를 제약합니다.”

## 코드 — 잔차 벡터가 만들어지는 곳

calibration\_pipeline/reprojection.py · residual\_vector · L336–351

```
target = state.cubes[int(obs.set_idx)]
if int(obs.cam) == self.gripper:
    if int(obs.event) not in self.robot_T:
        raise RuntimeError(f"robot FK missing for event {obs.event}")
    T_base_cam = np.asarray(self.robot_T[int(obs.event)], dtype=np.float64) @ state.gtc
else:
    if int(obs.cam) not in state.cams:
        raise RuntimeError(f"fixed camera {obs.cam} is not registered")
    T_base_cam = state.cams[int(obs.cam)]
prediction = project_points(
    inv_T(T_base_cam) @ target, obs.object_points,
    self.K[int(obs.cam)], self.D[int(obs.cam)],
)
chunks.append((prediction - np.asarray(obs.image_points).reshape(-1, 2)).reshape(-1))
return np.concatenate(chunks).astype(np.float64)
```

### 한 줄씩 읽기

- `obs.cam == gripper` → **손목카메라**:  
`robot_T[event] @ state.gtc` 즉  ${}^B T_G(e) {}^G T_{C_g}$
- `else` → **고정카메라**: `state.cams[cam]`
- `project_points(inv_T(T_base_cam) @ target, ...)`  
 →  $\pi(\cdot)$
- `prediction - obs.image_points` → **픽셀 잔차**
- `chunks` 에 계속 쌓아 `concatenate` → **모든 카메라·이벤트·마커가 하나의 벡터**

### 여기가 핵심

고정카메라와 손목카메라의 잔차가 **같은 리스트에 append** 된다.  
 이게 “eye-in-hand와 eye-to-hand를 하나의 문제로 푼다”는 말의 코드 수준 의미다.

# $K, D$ 는 고정인가 — 그리고 왜 중요한가

calibration\_pipeline/reprojection.py · project\_points · L143–152

```
def project_points(T_C_0: np.ndarray, object_points: np.ndarray,
                  K: np.ndarray, D: np.ndarray) -> np.ndarray:
    T_C_0 = np.asarray(T_C_0, dtype=np.float64)
    rvec = Rotation.from_matrix(T_C_0[:3, :3]).as_rotvec().reshape(3, 1)
    tvec = T_C_0[:3, 3].reshape(3, 1)
    projected, _ = cv2.projectPoints(
        np.asarray(object_points, dtype=np.float64), rvec, tvec,
        np.asarray(K, dtype=np.float64), np.asarray(D, dtype=np.float64),
    )
    return projected.reshape(-1, 2)
```

calibration\_pipeline/schema.py · L81–93 (최적화 변수가 아닌 입력들)

```
# Inputs that are constants, never optimization variables. In particular,
# intrinsics must not leak into stage 2 as an accidental extra degree of
# freedom. The observation mask is computed once, before any row is fit.
GLOBAL_FIXED_INPUTS = (
    "camera_intrinsics_K",
    "distortion_D",
    "target_geometry",
    "T_base_gripper_by_event",
    "train_test_split",
    "observation_quality_mask",
    "optimizer_loss_tolerances",
    "initialization_seeds",
)
```

## 답변 문장

“ $K_i, D_i$ 는 `project_points()` 에 상수로 전달되며 자유변수 목록에 없습니다. 즉 이 실험은 **intrinsic recalibration**이 아니라 extrinsic / hand-eye / target pose 추정 문제입니다.”

## Q5 예상 재질문

“ $K, D$ 를 고정했으면 **intrinsic** 오차가 전부 **extrinsic**으로 흡수되는 것 아니냐?”

→ 타당하고 사실이다. 대응: 사전 intrinsic calibration의 재투영 오차로 품질을 보고하고, 민감도 표에  $K, D$ 를 자유변수로 둔 조건을 하나 넣어 흡수량을 정량화하겠다.

## 왜 코드가 이걸 명시적으로 막아 두었나

주석: “*intrinsics must not leak into stage 2 as an accidental extra degree of freedom.*” 순차 조건의 2단계에서 실수로 자유도가 늘어나면 조건 간 비교가 깨진다.

# 순차(seq) vs 통합(U) — 무엇이 다른가

순차 —  $A0 \cdot A1 \cdot B1$

**1단계: eih 관측만**

${}^G T_{C_g}$ , target pose를 푼다

▼ 동결(freeze)

**2단계: e2h 관측만**

각  ${}^B T_{C_i}$ 만 푼다 · 카메라끼리 독립

1단계 오차가 2단계로 그대로 전파된다. e2h  $\rightarrow$  eih 되먹임(feedback)도, 교대 반복(alternating pass)도 없다.

통합 —  $A2 \cdot A3 \cdot A4 \cdot B2 \cdot B3$

$$\hat{\theta} = \arg \min_{\theta} \left[ \sum_{\mathcal{O}_{eih}} \rho(\|r\|^2) + \sum_i \sum_{\mathcal{O}_{e2h,i}} \rho(\|r\|^2) + E_{FK}(\theta) \right]$$

eih와 모든 e2h 원시 코너 잔차를 하나의 residual vector에 쌓아 한 번에 푼다. 한 카메라의 잔차가 공유 변수와 다른 카메라의 extrinsic을 함께 갱신한다.

**이것이 claim 1의 실험적 구현체**

“통합 정식화가 순차적 오차 누적을 제거한다”는 주장을 검증하는 축이 바로 seq  $\leftrightarrow$  U 비교다.

모든 시각 잔차에 동일하게 적용: `soft_l1`, `f_scale = 2 px`, `max_nfev = 300`, local SE(3) parameterization. 조건 간에 solver 설정 차이는 없다.

# 재투영 오차의 mm 표기를 지운 이유

“projection error를 왜 mm로 표시했는가”에 대한 답 — 네가 틀렸던 것을 인정하는 항목

## 기존 방식이 왜 틀렸나

픽셀 잔차를 정규화한 뒤 각 방법이 예측한 코너 깊이를 곱해 mm처럼 환산했다.

- 같은 픽셀 오차도 **초점거리·예측 깊이**에 따라 다른 mm가 된다
- 서로 다른 calibration 결과가 **단위 환산 기준까지 바꾼다**
- **깊이(광축) 방향 오차가 아예 빠져 있다** → 물리 오차로 읽으면 과소평가

## 깊이 방향 오차가 왜 안 보이는가



광축 위에서 앞뒤로 밀린 오차는 **같은 픽셀에 투영된다**. 그래서 픽셀 잔차 × 깊이는 3D 오차가 아니다.

## 답변 문장

“재투영의 직접 관측량은  $e_{\text{reproj}}^{\text{px}} = \sqrt{\frac{1}{2N} \sum_n ((\hat{u}_n - u_n)^2 + (\hat{v}_n - v_n)^2)}$ 이며 native distorted image에서 **px**만 보고합니다. 물리 단위 정확도는 **독립 측정한 3D GT에서만** 보고해야 합니다.”



PART 4 · 쟁점 4

# 모든 방법을 같은 데이터와 같은 기준으로 비교했는가?

전사 위치 42:38-49:47

“어떤 방법은 전체 데이터 적합값, 어떤 방법은 split 결과를 보고하면 **비교가 성립하지 않는다.**”

→ 방어 논리가 **가장 튼튼한 영역**이다.

# 왜 코너가 아니라 event 단위로 나누는가

이 설명을 못 하면 “그냥 랜덤 split 아니냐”로 끝난다

## 코너 단위로 나누면 — 누수

event 29 · cam0 영상

코너 절반 → train

event 29 · cam1 영상

코너 절반 → test

같은 순간 · 같은 큐브 위치  
test가 이미 train에서 본 정보를 담고 있음  
→ held-out이 held-out이 아니게 된다

## 현재 방식 — event 단위 · set별 층화

- 한 **이벤트**는 통째로 train 또는 test 중 하나에만 들어간다
- set(큐브를 놓아둔 구간)별로 **층화(stratify)** — 특정 set이 전부 test로 빠지는 일이 없다
- train에 eye-in-hand 큐브 이벤트가 최소 개수 남도록 보장 (`min_train_eih_cube_events`)

공통 seed

20260731

held-out

[29, 38, 46, 50, 53, 61, 69, 75, 92]

공유 항목

원시 검출 ·  $K, D$  · solver budget · seed · split · 사전 mask

## 답변 문장

“split은 코너 단위가 아니라 **event** 단위이고 **set**별로 층화했습니다. 같은 이벤트의 여러 카메라 영상이 train과 test로 갈라지면 **같은 순간의 정보가 양쪽에 새기** 때문입니다.”

# 코드 — split이 만들어지는 곳

calibration\_pipeline/table1.py · build\_event\_split · L323–333

```
events_by_set: Dict[int, set] = defaultdict(set)
eih_cube_by_set: Dict[int, set] = defaultdict(set)
for obs in observations:
    if obs.set_idx is None:
        continue
    s = int(obs.set_idx)
    events_by_set[s].add(int(obs.event))
    if obs.marker == "cube" and int(obs.cam) == int(gripper):
        eih_cube_by_set[s].add(int(obs.event))
    rng = np.random.default_rng(int(seed))
    train_events, test_events, eligible_sets = set(), set(), []
```

calibration\_pipeline/table1.py · L345–347 (고정 seed 셔플)

```
order = list(events)
rng.shuffle(order)
wanted = max(1, int(round(float(fraction) * len(events))))
```

calibration\_pipeline/table1.py · L360–363 (이벤트 통째로 배타 분할)

```
continue
eligible_sets.append(s)
test_events.update(chosen)
train_for_set = set(events) - set(chosen)
```

## 읽는 순서

- `events_by_set[s].add(event)` — set별로 이벤트를 모은다
- `default_rng(seed) + rng.shuffle(order)` — 고정 seed의 재현 가능한 셔플
- `wanted = round(fraction × len(events))` — set마다 비율만큼 test로
- `test_events.update(chosen) / train_for_set = set(events) - chosen` — 이벤트 통째로 배타 분할

## 중간의 next\_remaining 검사

test로 빼는 이벤트 때문에 train에 손목카메라 큐브 이벤트가 부족해지면 그 이벤트는 test로 빼지 않는다. 즉 split이 특정 조건을 학습 불가로 만들지 않도록 사전에 보호한다.

# ★ 절대 헛갈리면 안 되는 두 개의 표

board-only도 큐브가 포함된 공통 관측에서 다시 평가받는다

| 지표                                                | 의미                                                      | 순위 가능?                                                     |
|---------------------------------------------------|---------------------------------------------------------|------------------------------------------------------------|
| own Test px                                       | 각 방법이 실제 목적함수에 사용한 marker population에<br>서의 held-out 성능 | <b>불가</b><br>A0는 board만,<br>B2는 cube만 —<br>관측 집합 자체가<br>다름 |
| Common px<br>pixel transfer<br>$e_{\text{cross}}$ | 모든 방법에 동일한 관측 population을 적용                            | <b>가능</b><br>단, internal 한<br>정                            |

## 이 구분을 못 하면 무너지는 지점

교수님: “A0의 자체 Test 2.4675 px가 A3의 2.5315 px보다 작는데, 그럼 A0가 나은 거냐?”  
→ A0는 **보드만** 재투영한 값이고 A3는 **보드+큐브**다. 애초에 다른 문제를 푼 성적표라 직접 비교 불가. 같은 링(공통 관측)에서는 A3 **2.0105** < A0 **2.0193**.

## 공통 평가 population

|              |            |              |             |
|--------------|------------|--------------|-------------|
| <b>12</b>    | <b>229</b> | <b>6</b>     | <b>12</b>   |
| observations | corners    | camera pairs | transfer 방향 |

calibration\_pipeline/cross\_target.py · L165–176

```
[for method in METHOD_ORDER:
    print(f"[CROSS-TARGET] {method}")
    per_run[method] = [
        evaluate_common_target_run(
            run["transforms"], common_cameras, observations,
            prepared.board_initial, prepared.fixed_cubes,
            prepared.test_obs, prepared.robot_T, prepared.K_map,
            prepared.D_map, prepared.gripper,
            prepared.path_evaluation_mask)
        for run in stored_runs[method]
    ]
    summary.append(summarize(
```

## 여기도 정직하게

공통 관측 **12개**는 작다. 0.05 px 차이를 유의하다고 말하면 안 된다. → Part 8 Q2

# 평가는 동결(frozen)되어 있다 — 그리고 해시로 잠겨 있다

calibration\_pipeline/evaluation.py · L22–30 (px-only 계약)

```
REPROJECTION_METRIC_CONTRACT = {
    "primary_field": "rmse_px",
    "domain": "distorted_native_image_pixels",
    "residual": "predicted_corner_uv_minus_measured_corner_uv",
    "aggregation": "component_wise_RMSE_sqrt_sum_du2_dv2_over_2N",
    "parameters_frozen_during_evaluation": True,
    "model_dependent_observation_rejection": False,
    "millimetre_reprojection_conversion_reported": False,
}
```

calibration\_pipeline/path\_evaluation.py · L239–249 (mask 검증)

```
def validate_common_path_evaluation_mask(mask: Mapping) -> None:
    if mask.get("artifact_schema") != MASK_SCHEMA:
        raise ValueError("unknown path-evaluation mask schema")
    if mask.get("model_output_used_for_selection") is not False:
        raise ValueError("path-evaluation selection must be model independent")
    if mask.get("output_dependent_pose_gate") is not None:
        raise ValueError("output-dependent translation/rotation gates are forbidden")
    contract = mask.get("e_cross_contract", {})
    if contract != E_CROSS_CONTRACT:
        raise ValueError("e_cross is not the canonical fixed-camera cube consistency metric")
    if mask.get("e_cross_pixel_transfer_contract") != E_CROSS_PIXEL_TRANSFER_CONTRACT:
```

## 세 가지 금지 조항

- `parameters_frozen_during_evaluation: True` — 평가 중 재적합 없음
- `model_dependent_observation_rejection: False` — 결과를 보고 관측을 버리지 않음
- `millimetre_reprojection_conversion_reported: False` — mm 환산 금지 (쟁점 3)

그리고 mask 쪽에서 `model_output_used_for_selection` 이 False가 아니거나 `output_dependent_pose_gate` 가 존재하면 실행이 죽는다.

## 재현성 잠금

exact corner population · meta.json · intrinsics · pose manifest · 핵심 구현 파일의 **SHA-256**을 결과에 기록한다. 데이터나 코드가 바뀌면 옛 결과를 현재 결과처럼 쓸 수 없다.

baseline: c9ce0c90db9b12f6...391ea246

PART 5 · 쟁점 5

# OpenCV · 기존 연구와 비교했는가?

전사 위치 56:41-57:48

결론: 인정 항목. 다만 “안 했다”로 끝내지 말고 대상 목록과 연결 방식까지 같이 제시한다.

# ablation ≠ baseline — 지금 표가 답하는 질문의 범위

A5도 “외부 방법”이 아니라 독립 FK correction label을 위한 예약 행이다

## 지금 있는 것 — 내부 ablation

A0~A4 / B1~B3는 같은 파이프라인에서 구성요소를 빼거나 바꾼 조건들이다.

답하는 질문: “우리 파이프라인 안에서 cube · unified · hard/soft FK가 각각 어떤 영향을 주는가”

이 행들을 **OpenCV Tsai/Park/Horaud/Daniilidis**나 논문 **SOTA baseline**이라고 부르면 안 된다.

## 추후 붙일 것 — 외부 baseline 표

동일 조건으로 연결한 뒤 **별도 표**로 제시:

동일 event split

동일 raw detections

동일  $K, D$

동일 common mask

대상 목록 (이미 정해 둔 것):

OpenCV 표준 hand-eye 4종 · Tabb & Yousef 2017 · Allegro et al. 2024 · EasyHeC++ · Li et al. 2024 · Calib3R 2025 · Evangelista 2023

calibration\_pipeline/table1.py · L110–116 (A5는 미실행으로 명시)

```
PENDING_ROWS = {
    "A5": {
        "status": "not_run",
        "reason": "independent train-only 6-DoF FK correction labels are unavailable",
        "baseline": "same shared train-only reference state as A4",
    }
}
```

## 답변 문장

“지적이 맞습니다. 현재 표가 답하는 질문은 **내부 ablation**까지입니다. 외부 baseline은 **동일 event split · 동일 raw detections · 동일  $K, D$  · 동일 common evaluation mask**에 연결한 뒤 별도 표로 붙이겠습니다.”

PART 6 · 쟁점 6

# 아웃라이어는 정확히 어떻게 제거했는가?

전사 위치 27:39-34:09

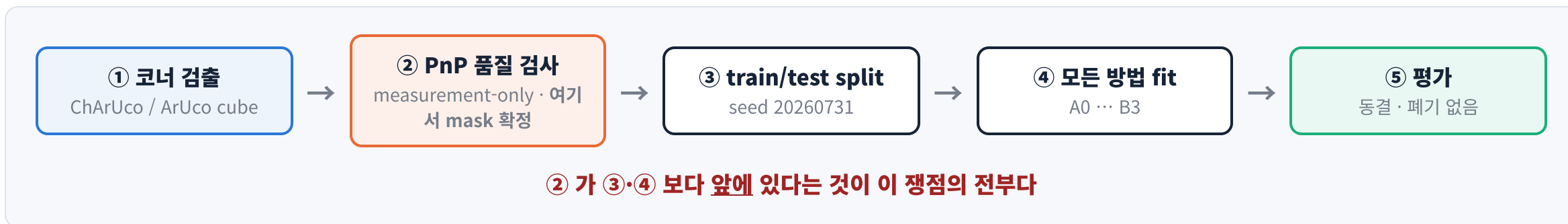
“최종 방법의 결과를 보고 outlier를 제거하면 **비교가 편향되지 않는가?**”

→ 방어 논리가 **가장 깔끔한 항목**이다.



# 핵심은 “언제” 거르느냐다

“결과를 보고 나쁜 점을 뺐다”는 의심에 대한 구조적 답



## ① 시점

mask 생성이 **split보다도**, 모든 **calibration fit보다도** 앞에 있다. measurement-only PnP 품질 검사 1회.

## ② 공유

모든 방법이 같은 **mask**를 쓴다. 방법 출력으로는 선택 불가 — 코드가 `ValueError` 로 죽는다.

## ③ 점수 계산 ★

RANSAC은 초기화에만 쓰고, accept/reject 점수는 검출된 전체 코너로 계산한다.

## 이대로 말하면 되는 문장

“현재 outlier 처리는 **최적화 결과를 보고 유리한 점을 제거하는 방식이 아닙니다**. calibration method를 실행하기 **전에** 고정된 PnP 규칙으로 동일 mask를 만들고 모든 방법이 그 mask를 공유합니다. non-planar cube의 RANSAC은 초기 pose 추정에만 쓰이고, 최종 accept/reject 점수는 **전체 검출 코너**로 계산하므로 RANSAC이 나쁜 코너를 점수에서 숨길 수 없습니다.”

## 코드 — 판단 기준이 “전체 코너”라는 증거

calibration\_pipeline/cube\_detection.py · \_pnp\_quality docstring · L97–103

```
"""Estimate a measurement-only pose and score every detected corner.

Planar support uses IPPE and explicitly chooses the positive-depth solution
with the lowest all-corner error. Non-planar support uses RANSAC only to
initialize the pose; acceptance is still based on the RMSE of *all* input
corners, so an outlier cannot disappear from the common quality mask.
"""
```

calibration\_pipeline/cube\_detection.py · L174–181 (전체 코너로 채점)

```
projected, _ = cv2.projectPoints(obj, rvec, tvec, K, D)
errors = np.linalg.norm(projected.reshape(-1, 2) - img, axis=1)
if not np.all(np.isfinite(errors)):
    continue
rmse = float(np.sqrt(np.mean(np.square(errors))))
inlier_fraction = float(np.mean(errors <= threshold_px))
scored.append((rmse, inlier_fraction, solver))
return min(scored, key=lambda item: item[0]) if scored else None
```

calibration\_pipeline/cube\_detection.py · L131–140 (RANSAC은 초기화용)

```
ok, rvec, tvec, inliers = cv2.solvePnP(
    obj, img, K, D,
    iterationsCount=200,
    reprojectionError=threshold_px,
    confidence=0.999,
    flags=cv2.SOLVEPNP_EPNP,
)
if ok:
    solver = "RANSAC-EPNP"
    if inliers is not None and len(inliers) >= 4:
```

calibration\_pipeline/cube\_detection.py · L294–297 (threshold 적용)

```
pnp_rmse, pnp_inlier_fraction, pnp_solver = pnp_quality
if pnp_rmse > float(max_error):
    pnp_error_rejections += 1
    continue
```

### ★ 이 쟁점 답변의 전부인 한 문장

“RANSAC이 나쁜 코너를 점수 계산에서 숨길 수 없다.”

errors 는 inliers 가 아니라 img (전체 검출 코너) 기준이다.

# 수식과 임계값, 그리고 실제로 몇 개나 버렸는가

accept/reject 점수 —  $N$ 은 검출된 전체 코너 수

$$e_{\text{PnP}} = \sqrt{\frac{1}{N} \sum_{n=1}^N \|\hat{\mathbf{u}}_n - \mathbf{u}_n\|_2^2}$$

| 대상         | PnP 초기화                                        | 임계값        |
|------------|------------------------------------------------|------------|
| 고정카메라 큐브   | planar → IPPE<br>non-planar → RANSAC-EPNP(+LM) | 3 px       |
| 그리퍼카메라 큐브  | 동일                                             | 5 px       |
| ChArUco 보드 | dictionary/board ID 제약 · 최소 4 코너               | 별도 gate 없음 |

## planar일 때 한 가지 더

IPPE는 해가 여러 개 나온다. 그중 모든 점이 **positive depth**이고 전체 코너 **RMSE가 최소**인 해를 고른다 — 결과를 보고 고르는 게 아니라 **관측만으로** 고른다.

## session02 진단

|                    |           |
|--------------------|-----------|
| 읽은 cube image      | 78        |
| PnP 자체 실패          | 0         |
| RMSE threshold 탈락  | 1         |
| <b>최종 accepted</b> | <b>77</b> |
| accepted RMSE 중앙값  | 1.0847 px |
| accepted RMSE 최대값  | 3.2187 px |

최대값이 3 px를 넘는 이유: **그리퍼카메라에는 5 px 기준**이 적용되기 때문.

## 가장 강한 반박 근거

**78개 중 1개만 버렸다.** “유리하게 걸렸다”는 의심을 숫자 하나로 무력화한다. 아무것도 안 버린 것과 사실상 같은 상태에서 나온 결과다.

PART 7 · 추가 발견

# pose 좌표계 혼용

## — 네가 먼저 꺼내야 하는 항목

교수님 질문에 답하려고 코드를 추적하다가 발견한 문제.

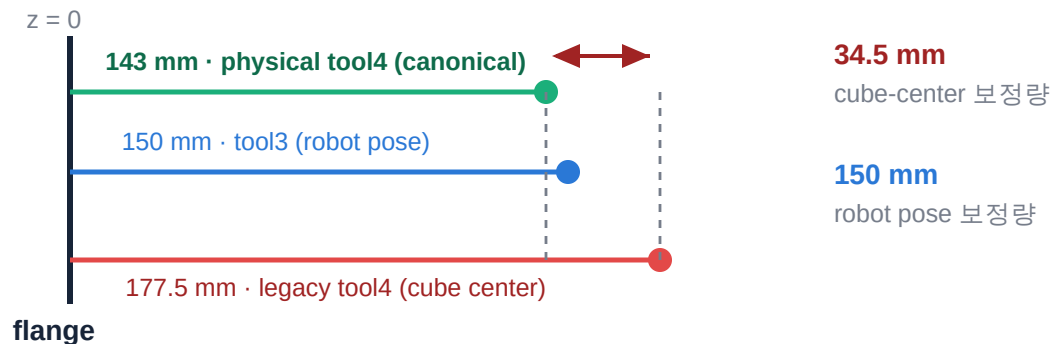
성능 튜닝이 아니라 문제의 well-posedness에 관한 것이다.

# 같은 필드에 서로 다른 좌표계가 섞여 있었다

| event   | robot pose 기준      | cube center 기준          |
|---------|--------------------|-------------------------|
| 0 ~ 89  | tool3 (150 mm TCP) | legacy tool4 (177.5 mm) |
| 90 ~ 95 | flange (0 mm)      | physical tool4 (143 mm) |

## 왜 치명적인가

단일 hand-eye 변환 하나가 두 개의 서로 다른 프레임을 동시에 설명해야 한다. 이걸 최적화가 잘 안 되는 문제가 아니라 일관된 해가 존재하지 않는 문제다.



정규화 — right-multiplication correction

$$T_{\text{canonical}}^B = T_{\text{reported}}^B T_{\text{canonical}}^{\text{reported}}$$

pose\_convention\_manifest.json · canonical frame 선언

```
"schema_version": "pose_convention_manifest_v1",
"canonical_robot_frame": "robot_flange",
"canonical_cube_center_frame": "physical_cube_center_tool4_143mm",
```

pose\_convention\_manifest.json · L13~18 event 0~89 robot 보정 (-0.1500 m)

```
"T_reported_robot_to_canonical": [
  [1.0, 0.0, 0.0, 0.0],
  [0.0, 1.0, 0.0, 0.0],
  [0.0, 0.0, 1.0, -0.1500],
  [0.0, 0.0, 0.0, 1.0]
],
```

## 중요 — 순환을 다시 만들지 않기

convention 선택에 영상이나 fitted calibration 결과를 사용하지 않는다.  
manifest에 사전 선언된 물리적 오프셋만 쓴다.

# 코드와 결과 — 그리고 왜 먼저 보고해야 하는가

calibration\_pipeline/pose\_convention.py · L240–249 (정규화 적용)

```
{capture["canonical_robot_pose_matrix_4x4"] = (
    robot @ robot_correction).tolist()
if set_cube is not None:
    capture["canonical_set_cube_center_matrix_4x4"] = (
        set_cube @ cube_correction).tolist()
if capture_cube is not None:
    capture["canonical_capture_cube_center_matrix_4x4"] = (
        capture_cube @ cube_correction).tolist()
capture["pose_convention_segment"] = int(segment_index)
normalized_events.append(event)
```

calibration\_pipeline/pose\_convention.py · L253–263 (단일 signature 강제)

```
clusters = _cluster_relative_signatures(
    canonical_rows,
    "canonical_robot_pose_matrix_4x4",
    "canonical_capture_cube_center_matrix_4x4",
    float(manifest.get("relative_signature_translation_tol_mm", 2.0)),
    float(manifest.get("relative_signature_rotation_tol_deg", 0.2)),
)
if len(clusters) != 1:
    raise ValueError(
        "pose convention normalization did not produce one robot-to-cube-centre "
        f"signature: {[cluster['events'] for cluster in clusters]}")
```

## 정규화 후 검증 결과 (set 11)

**0.679**

mm · cube center 최대 편차

**0.000236**

deg · 회전 편차

단일 robot-to-cube-center signature 검사, set stationarity 검사 모두 통과. 현재 표의 모든 숫자는 이 정규화 이후 다시 생성한 값이다.

## ★ 왜 먼저 꺼내야 하나

이건 “이전 결과가 왜 이상했는지”의 실제 원인일 가능성이 높다. 교수님이 나중에 발견하면 신뢰도 타격이 크고, 자진 보고하면 “코드를 끝까지 추적했다”는 증거가 된다.

## 덧붙일 안전장치

manifest 없이 서로 다른  $\text{inv}(T_{\text{robot}}^B) T_{\text{cube}}^B$  signature가 2개 이상 검출되면 fit 전에 실행이 중단된다.

PART 8 · 면담 대비

# 전체 결과 · 예상 재질문 · 말할 순서

여기서부터는 이해가 아니라 **실전** 준비다.

**Q1** 하나만 **확실히** 준비하면 나머지는 앞의 내용으로 커버된다.

# canonical 결과 전체 — session02

각 값은 3개 초기화의 평균 · 동일 split · 동일 mask · 동일 solver

| ID | 구성                     | 상태        | Train px      | 자체 Test px    | 픽셀 전이 px      | $e_{\text{cross}}$ mm | $e_{\text{cross}}$ deg | $e_{\text{e2e}}$ mm | 공통 px         |
|----|------------------------|-----------|---------------|---------------|---------------|-----------------------|------------------------|---------------------|---------------|
| A0 | 보드 · 순차 · vision       | complete  | 3.8654        | 2.4675        | 11.7971       | 14.2795               | 0.8603                 | 2.9446              | 2.0193        |
| A1 | 큐브+보드 · 순차             | complete  | 3.7146        | 2.9942        | 12.7282       | 15.8616               | <b>0.8086</b>          | 3.7887              | 2.0975        |
| A2 | 큐브+보드 · 통합             | complete  | 3.7118        | 2.9715        | 12.0222       | 14.8495               | 0.8437                 | 3.4324              | 2.0628        |
| A3 | 큐브+보드 · 통합 · FK-fixed  | complete  | <b>3.9662</b> | <b>2.5315</b> | <b>9.6520</b> | <b>11.5631</b>        | <b>0.8442</b>          | <b>3.1686</b>       | <b>2.0105</b> |
| A4 | 큐브+보드 · 통합 · soft FK   | preflight | 3.7115        | 2.9525        | 11.9717       | 14.7545               | 0.8466                 | 3.3922              | 2.0592        |
| A5 | 독립 FK correction label | not run   | —             | —             | —             | —                     | —                      | —                   | —             |
| B1 | 큐브+보드 · 순차 · soft FK   | preflight | 3.7136        | 2.9729        | 12.6368       | 15.7124               | 0.8119                 | 3.7334              | 2.0903        |
| B2 | 큐브만 · 통합 · soft FK     | preflight | 3.0497        | 4.2202        | 11.1316       | 14.0157               | 1.0872                 | 4.1085              | 2.9214        |
| B3 | 보드 · 통합 · vision       | complete  | 3.8654        | 2.4675        | 11.7964       | 14.2776               | 0.8608                 | 2.9440              | 2.0195        |

## 표를 볼 때 반드시 같이 말할 것

① Train px와 자체 Test px는 조건 간 순위에 쓸 수 없다 (marker population이 다름) · ② A4/B1/B2는 **preflight** · ③ 회전 지표는 A1/B1이 더 낮다 · ④ 각 숫자는 3개 초기화의 평균



# Q1 — 가장 위험한 질문

**“재투영은 2 px인데 카메라 간 pixel transfer는 9.65 px,  $e_{\text{cross}}$  병진은 11.56 mm? 4~5배 차이는 뭐냐”**

이게 시스템의 진짜 성능이 11 mm급이라는 뜻으로 읽힐 수 있다. 메모리에 있는 SOTA 기준(EasyHeC++ 1.35 mm, Li et al. 0.93 mm)과 **10배 차이**라 교수님도 알고 물어보는 것.

## 왜 격차가 나는가 — 대응 논리

- 개별 재투영 2 px은 각 카메라의 extrinsic이 자기 관측에 맞춰진 결과라 구조적으로 낙관적이다
- transfer /  $e_{\text{cross}}$ 는 두 카메라 extrinsic의 상대 오차가 드러난다
- cube PnP의 깊이 방향 오차가 baseline 길이만큼 각도로 증폭된다 (마커 22 mm는 작다)

이 격차 자체를 “재투영 오차만 보면 안 된다”는 네 주장의 증거로 쓸 수 있다.

## 절대 하면 안 되는 것

11.56 mm를 **얼버무리기**. 숫자를 피하면 “정리가 안 됐다”가 아니라 “숨긴다”로 읽힌다.

## 이대로 말하면 되는 문장

“현재 internal cross-camera 불일치가 **11.5 mm 수준**이고, **이건 목표 수준이 아니라는 것을 인지하고 있습니다**. 원인 분해를 다음 단계로 잡았습니다 — ① intrinsic 품질, ② 큐브 마커 크기 22 mm, ③ baseline·시야각 배치, ④ 코너 검출 정밀도.”

## 준비해 두면 좋은 것

- 사전 intrinsic calibration의 재투영 RMSE 값
- 카메라 쌍별 baseline 길이와 큐브까지의 거리
- $e_{\text{cross}}$ 를 카메라 쌍별로 분해한 값 (특정 쌍이 지배하는지)

## Q2 ~ Q4

### Q2. “공통 관측이 12개, 코너 229개? 이걸로 방법 순위를 매긴다고?”

A3 2.0105 vs A2 2.0628 — 차이 0.05 px가 표본 12개에서 유의한가? 거의 확실히 아니다.

→ “현재 common evaluation의 표본이 작아 조건 간 차이를 유의하다고 주장하지 않습니다. 다음 촬영에서 board 가시 데이터와 큐브 회전 다양성을 확보해 공통 관측 수를 늘리고, **k-fold + 신뢰구간**으로 보고하겠습니다.”

### Q3. “FK covariance를 실측 못 했으면 A4/B1/B2 숫자는 왜 표에 있냐”

`preflight_simulation_prior` 로 라벨링돼 있고 **확증 결과로 쓰지 않는다**. 실측은 반복 측정  $N \geq 7$  (6-DoF 표본공분산 full rank 조건)로 취득 예정.

→ 취득 프로토콜을 구체적으로 말하면 훨씬 좋다: 같은 joint command 반복, 몇 개 pose에서, 몇 회.

### Q4. “그럼 결국 A3가 제일 좋다는 거냐, 아니냐”

흐릿하게 답하면 “정리가 안 됐다”는 인상을 준다. 한 문장으로 준비해 둘 것:

“internal pixel·병진 기준에서는 A3가 가장 강하지만, 회전 지표는 혼재하고 외부 GT가 없어 최종 채택은 보류입니다. 외부 GT와 실측 FK covariance 이후에 결정합니다.”

## Q5 ~ Q7 · 그리고 미리 정리해 둘 불일치

### Q5. “ $K, D$ 를 고정했으면 intrinsic 오차가 전부 extrinsic으로 흡수되는 것 아니냐”

타당하고 **사실이다**. → intrinsic은 사전 calibration 재투영 오차로 품질 보고 + 민감도 표에  $K, D$  자유 조건을 하나 넣어 흡수량을 정량화하겠다.

### Q6. “board는 왜 outlier gate가 없냐”

ChArUco는 dictionary/board ID 제약 + 최소 4코너가 이미 강한 구조적 필터라 별도 gate를 두지 않았고, 큐브처럼 PnP RMSE gate가 필요한지는 검토 중. 문서에 “board 전용 homography gate를 추가한 게 아니다”라고 이미 명시해 두었으므로 은폐가 아니다.

### Q7. “A5는 왜 비어 있고 B3는 뭐냐”

표에 있는데 설명 못 하는 행이 있으면 인상이 나쁘다. 면담 전에 B1/B2/B3 정의를 한 줄씩 정리해 둘 것.

B1 = 순차 + soft FK (A4의 -Unified) · B2 = 큐브만 + soft FK (A4의 -board) · B3 = 보드만 + 통합 (-cube)

### ⚠ 미리 정리하고 들어갈 불일치

이전 미팅 기준으로 A4 = “A2@ $\lambda=3$  soft FK anchor + Ridge 잔차보정”으로 명명했었다.

그런데 현재 문서에서 A4는 “corrected-FK-factor”로 정의돼 있고, Ridge 잔차보정 행이 보이지 않는다.

교수님이 이전 노트와 대조하면 “A4가 언제 바뀌었냐”가 나올 수 있다.

#### 확인할 것

- ① A4 정의가 바뀐 거면 → 왜 바뀌었는지 한 줄
- ② Ridge 잔차보정이 별도 행에서 빠진 거면 → 어디로 갔는지

# 말할 순서 — 6단계

## 1. 쟁점 1 먼저 인정 — 외부 GT 없음, 절대정확도 주장 안 함

→ 신뢰 확보. 방어부터 시작하면 전체가 변명으로 들린다

## 2. 쟁점 7 자진 보고 — pose convention 혼용 → canonical 확정

→ “코드를 끝까지 봤다”는 증명

## 3. 쟁점 3 — 목적함수 · 변수 · 단위 정의, mm 표기 제거

→ 기술적 명확성

## 4. 쟁점 4 · 6 — 동일 split · 동일 mask · 사전 outlier 제거

→ 공정성 방어. 여기가 가장 튼튼하다

## 5. 쟁점 2 — FK 순환을 $e_{\text{cross}}$ 로 반박

→ 핵심 논거. 앞의 4단계가 있어야 설득력이 생긴다

## 6. 쟁점 5로 마무리 — 외부 baseline 계획 + 대상 목록 제시

→ “추후 진행”이 회피가 아니라 계획으로 읽히게

### 포지션 정리

|         |                             |
|---------|-----------------------------|
| 튼튼한 것   | 쟁점 4 · 6 (동일 기준 · 사전 mask)  |
| 날카로운 것  | 쟁점 2 ( $e_{\text{cross}}$ ) |
| 인정할 것   | 쟁점 1 · 5 (외부 GT · baseline) |
| 먼저 꺼낼 것 | 쟁점 7 (pose convention)      |

### 준비 우선순위

Q1 (11.56 mm) 하나만 확실히 준비하면 나머지는 이 자료 내용으로 다 커버된다.

# 한 장 요약 — 이것만 외우고 들어가도 된다

| # | 쟁점              | 핵심 답변 한 줄                                                  |
|---|-----------------|------------------------------------------------------------|
| 1 | 외부 GT           | 없다. 그래서 절대정확도로 기술 금지 플래그를 코드에 박았다.                         |
| 2 | FK 순환           | FK 사용 강도를 축으로 만들고, FK가 전혀 없는 $e_{\text{cross}}$ 로 다시 평가했다. |
| 3 | 목적함수 · 단위       | PnP 평균이 아니라 원시 코너 픽셀 잔차 하나의 벡터. $K, D$ 는 상수. mm 환산은 제거.    |
| 4 | 동일 기준           | event 단위 증화 split, 동일 mask, 평가 동결, SHA-256 잠금.             |
| 5 | 외부 baseline     | 지금은 내부 ablation까지. 동일 조건으로 연결해 별도 표로 붙인다.                  |
| 6 | 아웃라이어           | fit 이전의 공통 PnP mask. 점수는 RANSAC inlier가 아니라 전체 코너. 78→77.  |
| 7 | pose convention | tool3/tool4 혼용을 manifest로 정규화. 편차 0.679 mm.                |

## 최종 요약 발언 (그대로 읽어도 됨)

“목적함수를 원시 코너의 pixel reprojection으로 통일했고,  $K, D$ 는 고정한 상태에서 eye-in-hand와 eye-to-hand 관측을 공유 SE(3) 변수로 결합했습니다. 재투영 오차를 임의로 mm로 환산하던 표기는 제거했습니다. 비교는 event 단위의 동일 held-out split, 동일 관측·solver·seed·사전 고정 mask에서 다시 수행했습니다. FK-fixed 방법은 FK가 들어간 closure 오차만으로 평가하지 않고, FK를 사용하지 않는 고정카메라 간 pixel transfer와  $e_{\text{cross}}$ 도 함께 평가했습니다. 현재 A3가 내부 pixel·병진 지표에서 가장 강하지만 회전 지표는 혼합되어 있고, 외부 GT가 없기 때문에 절대 정확도 우위로 주장하지 않습니다. Soft-FK는 실측 covariance 전이라 preflight로만 표시합니다. Cube outlier는 fit 전 PnP 규칙으로 모든 방법에 동일하게 제거하며, 최종 accept/reject는 RANSAC inlier가 아니라 전체 검출 코너 RMSE로 판단합니다. 외부 물리 GT 검증과 SOTA baseline 비교는 추후 진행할 예정입니다.”

# 준비 끝. Q1만 챙기면 된다.

튼튼한 것 쟁점 4 · 6 | 날카로운 것 쟁점 2 | 인정할 것 쟁점 1 · 5 | 먼저 꺼낼 것 쟁점 7

남은 준비 항목 세 가지

- ①  $e_{\text{cross}}$  11.56 mm 원인 분해 (intrinsic / 마커 크기 / baseline / 코너 정밀도)
- ② B1 · B2 · B3 정의 한 줄씩
- ③ A4 정의 변경 이력과 Ridge 잔차보정의 현재 위치

canonical: CP\_result/session02/late\_table1/TABLE1\_RESULTS.md

조건별 해석: session02\_result\_table1.md